

Unit 1: Computational Thinking & Algorithmic Logic • Topic 3

Flowcharts, Pseudocode / Tracing / Dry Run

Representing algorithms visually (flowcharts) and in structured text (pseudocode), and verifying them by hand (tracing).

An algorithm written in plain numbered sentences (Topic 2) is correct, but it can still be hard to visualise or verify. **Flowcharts** and **pseudocode** are two standard tools that represent an algorithm more precisely — one visually, one in code-like text — and **tracing / dry running** is how we verify either of them by hand before typing any real code.

3.1 Flowcharts

A **flowchart** is a diagram that represents an algorithm using standard geometric symbols connected by arrows, showing the exact order in which steps are executed. Because it is visual, a flowchart often makes the logic of a program — especially decisions and loops — easier to follow than plain text.

Standard Flowchart Symbols

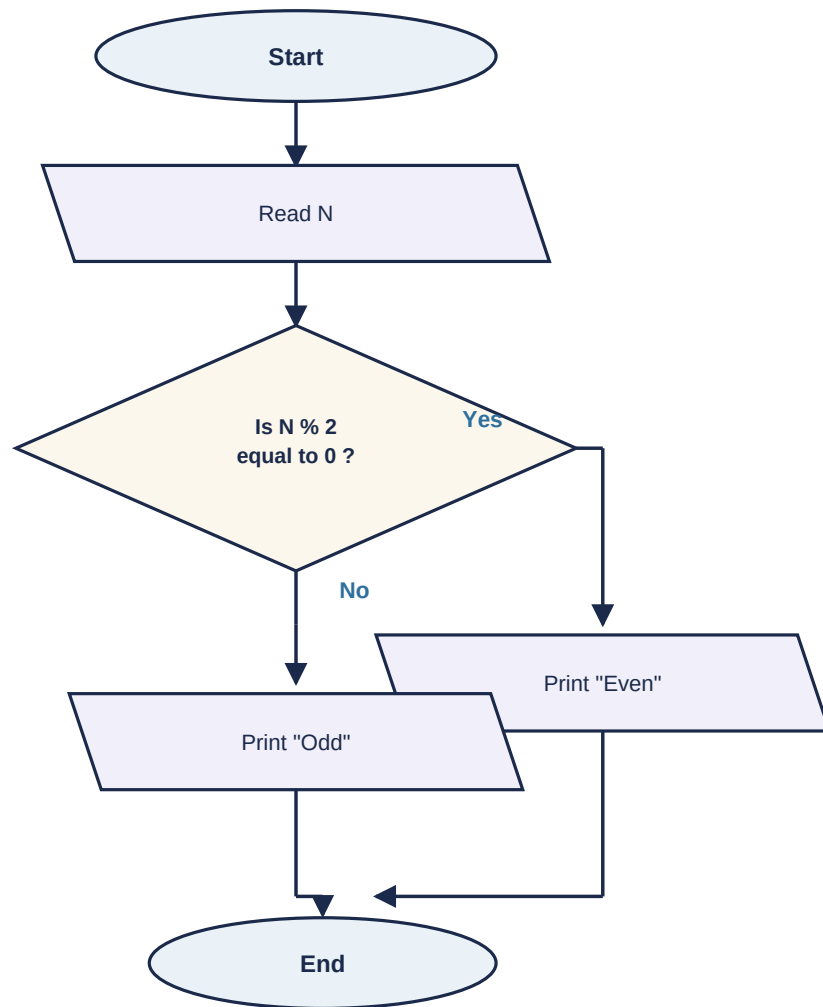
Symbol	Shape	Meaning
Terminal	Oval / Ellipse	Marks the Start or End of the flowchart
Input / Output	Parallelogram	Reading input or displaying/printing output
Process	Rectangle	A processing step, e.g. a calculation or assignment
Decision	Diamond (Rhombus)	A yes/no or true/false question with two (or more) exits
Flow line	Arrow	Shows the direction/order of execution
Connector	Small circle	Joins two parts of a flowchart, e.g. across a page break

Rules for Drawing a Good Flowchart

- Every flowchart must have exactly one Start and at least one End symbol.
- Flow lines (arrows) should generally go top-to-bottom or left-to-right.
- A decision (diamond) must have exactly two outgoing paths, usually labelled Yes/No or True/False (more for a multi-way decision).
- Keep the text inside each symbol brief and clear.
- Use connectors (circles) if a flowchart is too large for one page instead of crossing lines messily.

Worked Example: Flowchart to Check Even or Odd

The flowchart below represents this simple algorithm: read a number N; if N is exactly divisible by 2, it is Even, otherwise it is Odd.



3.2 Pseudocode

Pseudocode is a plain, structured, language-independent way of writing an algorithm that looks similar to real code (using words like IF, ELSE, WHILE, READ, PRINT) but ignores the strict syntax rules of any particular programming language. It is a bridge between a plain-English algorithm and actual Python code.

Common Pseudocode Conventions

Keyword	Used for
BEGIN / END	Start and end of the algorithm or a block
READ / INPUT	Taking a value from the user
PRINT / DISPLAY / OUTPUT	Showing a result
IF ... THEN ... ELSE	Decision-making
WHILE ... DO / FOR ... DO	Repetition (loops)
SET x = value / x ← value	Assigning a value to a variable

Worked Example: Pseudocode for Even/Odd Check

```

BEGIN
  READ N
  IF N MOD 2 = 0 THEN
    PRINT "Even"
  ELSE
    PRINT "Odd"
  END IF
END

```

Notice how closely this maps onto the flowchart above — and how close it is to real Python. This is exactly why pseudocode is written before code: it lets you focus purely on logic.

Flowchart vs Pseudocode vs Python Code

Aspect	Flowchart	Pseudocode	Python Code
Form	Diagram / symbols	Structured plain text	Actual syntax
Best for	Visualising flow & logic	Detailed step-by-step logic	Running on a computer
Rules	Loose (symbol conventions)	Loose (no strict grammar)	Strict (Python syntax)

3.3 Tracing / Dry Run

Tracing (also called a **dry run**) means manually walking through an algorithm, flowchart, or pseudocode *step by step*, on paper, using sample input values — without running it on a computer — and noting how every variable changes at each step. It is one of the most effective ways to catch logic errors before writing (or while debugging) real code.

Why Dry Run Matters

- It verifies an algorithm's correctness before time is spent coding it.
- It reveals logic errors that syntax checking cannot catch (e.g. an off-by-one loop bound).
- It builds the habit of predicting a program's output — an essential debugging skill.
- It is often asked directly in exams as a 'trace table' question.

Worked Example: Dry Run of the Sum-of-N-Numbers Algorithm

Recall the algorithm from Topic 2: *set sum = 0, i = 1; while i ≤ N: sum = sum + i, i = i + 1; print sum*. Let us dry run it for N = 4 using a trace table:

Step	Condition ($i \leq N$)	sum	i
Initial	—	0	1
Iteration 1	$1 \leq 4 \rightarrow \text{True}$	$0 + 1 = 1$	2
Iteration 2	$2 \leq 4 \rightarrow \text{True}$	$1 + 2 = 3$	3
Iteration 3	$3 \leq 4 \rightarrow \text{True}$	$3 + 3 = 6$	4
Iteration 4	$4 \leq 4 \rightarrow \text{True}$	$6 + 4 = 10$	5
Check	$5 \leq 4 \rightarrow \text{False}$ — loop ends	10	5

The algorithm correctly prints **10** (1+2+3+4), confirming it is correct for this test case.

TIP: Always dry run with at least one normal case, one boundary case (e.g. $N = 0$ or $N = 1$), and, where relevant, one invalid case — a single test case is rarely enough to trust an algorithm.

Practice Questions

A. Multiple Choice Questions

- Which flowchart symbol is used to represent a decision?
(A) Rectangle (B) Oval (C) Diamond (D) Parallelogram
- In a flowchart, the oval/ellipse symbol represents:
(A) Input/Output (B) Process (C) Start/End (D) Decision
- Pseudocode is best described as:
(A) Actual Python code (B) A diagram of symbols (C) Plain structured text resembling code, with no strict syntax (D) Machine code
- Manually walking through an algorithm step by step with sample data is called:
(A) Compiling (B) Debugging the syntax (C) Tracing / Dry Run (D) Installing
- A decision symbol in a flowchart normally has how many outgoing paths?
(A) One (B) Two (C) Four (D) Zero

B. True / False

- A flowchart can have more than one Start symbol.
- Pseudocode must strictly follow Python's exact syntax rules.
- A trace table helps verify an algorithm's correctness before coding it.
- The parallelogram symbol in a flowchart is used for input and output operations.

C. Short Answer Questions

- List the standard flowchart symbols and what each one represents.
- What is pseudocode, and why is it useful before writing actual code?
- Define tracing/dry run and explain why it is done before coding.
- State any three rules that should be followed while drawing a flowchart.

D. Long Answer / Descriptive Questions

- Draw (describe symbol-by-symbol, in words) the flowchart for an algorithm that reads three numbers and prints the largest of them.
- Write pseudocode for an algorithm that reads N numbers one by one and counts how many of them are positive, negative, and zero. Then dry run it for the inputs: 5, -3, 0, 8, -1 using a trace table.

E. Programming Problems

- Draw a flowchart and write pseudocode for calculating the simple interest given principal (P), rate (R) and time (T), using the formula $SI = (P \times R \times T) / 100$.
- Write pseudocode for an algorithm that finds the factorial of a number N, then create a trace table for N = 5, showing how the product changes at every step.
- Draw a flowchart for an algorithm that checks whether a given number is positive, negative, or zero.

Answer Key (A & B): MCQ: 1-C, 2-C, 3-C, 4-C, 5-B | T/F: 6-False, 7-False, 8-True, 9-True